

Lab 07: Rent Due, Lights Due, Mountain Dew – SOLUTIONS

PSTAT 100, Summer Session A 2025 with Ethan P. Marzban

MEMBER 1 (NetID 1) MEMBER 2 (NetID 2)
MEMBER 3 (NetID 3)

July 17, 2025

Required Packages

```
library(ottr)      # for checking test cases (i.e. autograding)
library(pander)    # for nicer-looking formatting of dataframe outputs
library(tidyverse) # for graphs, data wrangling, etc.
```

Logistical Details

Logistical Details

- This lab is due by **11:59pm on Friday, July 18, 2025**.
- Collaboration is allowed, and encouraged!
 - If you work in groups, list ALL of your group members' names and NetIDs (not Perm Numbers) in the appropriate spaces in the YAML header above.
 - Please delete any “MEMBER X” lines in the YAML header that are not needed.
 - No more than 3 people in a group, please.
- Ensure your Lab properly renders to a **.pdf**; non-**.pdf** submissions will not be graded and will receive a score of 0.
- Ensure all test cases pass (test cases that have passed will display a message stating "All tests passed!")

Lab Overview and Objectives

Welcome to another PSTAT 100 Lab! In this lab, we will cover the following:

- Linear Regression (both simple and multiple)
- Multicollinearity

Part I: A Mini-Project

In this portion of the lab, we'll undergo a sort of mini-project to explore the implementation of **linear regression** in R.

Recap: Statistical Models

Given data $\mathcal{D} := \{\vec{x}_i, y_i\}_{i=1}^n$ consisting of observations y_i of a **response variable** y and observations $\vec{x} := (x_{i1}, \dots, x_{ip})$ on p **explanatory** or **predictor variables** x_1 through x_p , a **statistical model** assumes the relationship between y_i and $\vec{x}_i$ to be

$$y_i = f(\vec{x}) + \varepsilon_i$$

for some **noise** term ε_i .

A **linear regression** model assumes:

- 1) A linear signal function; i.e. $f(\vec{x}_i) = \beta_0 + \sum_{j=1}^p x_{ij}$
- 2) Numerical response values (i.e. y is assumed to be a numerical variable as opposed to a categorical one).

The matrix representation of a linear regression model is:

$$\underbrace{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}}_{:=\vec{y}} = \underbrace{\begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}}_{:=\mathbf{X}} \underbrace{\begin{pmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix}}_{:=\vec{\beta}} + \underbrace{\begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}}_{:=\vec{\varepsilon}}$$

It is typical to assume i.i.d. Gaussian errors:

$$\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2) \iff \vec{\varepsilon} \sim \mathcal{N}_n(\vec{0}, \sigma^2 \mathbf{I})$$

The **ordinary least squares** (OLS) fit to the data seeks to find estimators $\hat{\beta}$ that solve the following minimization problem:

$$\hat{\beta} = \arg \min_{\vec{b}} \left\{ \|\vec{y} - \mathbf{X}\vec{b}\|^2 \right\}$$

which, under certain conditions, admits the following solution:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \vec{y}$$

In R, the function `lm()` performs model fitting of an MLR model to the data assuming L_2 loss.

Introduction to the Dataset

In this part of the lab, we consider a simulated dataset containing information on 302 different rental properties. For each rental property, we have observations on:

- **ppm**: the price per month (i.e. rental costs), in thousands of dollars
- **num_bedrooms**: the number of bedrooms (a value of 0 indicates a studio apartment)
- **dist_to_highway**: the distance (in miles) to the nearest highway
- **dist_to_town_center**: the distance (in miles) to the nearest town center

The data is stored in the `rental.csv` file, located in the `data/` subfolder.

Our ultimate goal is to model the rent (`ppm`) as a function of the other three variables included in the dataset.

Tip

If `df` is the name of an R dataframe and `y` is the name of one of the columns in `df`, running

```
lm(y ~ ., data = df)
```

is a shorthand to regress `y` onto *all* other columns present in `df`.

Question 1

Regress `ppm` onto all of the other three variables present in the dataset (`num_bedrooms`, `dist_to_highway`, and `dist_to_town_center`). Assign this model to an R object called `full_mod`.

Solution:

```
## replace this line with your code
```

Answer Check:

```
# DO NOT EDIT THIS LINE
invisible({check("tests/q1.R")})
```

Test q1 failed:

```
object 'full_mod' not found
```

! Question 2

Produce a residuals plot of the fit from Question 1, and comment on the fit. You may want to look up the help file for `lm()` to see how to extract the residuals and fitted values.

Solution:

```
## replace this line with your code
```

There are no obvious signs of trend or heteroskedasticity; hence, this is indicative of a model that is fitting relatively well.

Answer Check:

There is no autograder for this question; your TA will manually check that your answers are correct.

! Question 3**Part (a)**

Produce a regression table from the `full_mod` fit, and interpret the estimated coefficient values in the context of the problem. For example, what average change in rent is expected from a one-unit change in the number of bedrooms, holding all else constant? What about a one-unit change in the distance to the nearest town center? etc.

Solution:

```
## replace this line with your code
```

Answer Check:

There is no autograder for this question; your TA will manually check that your answers are correct.

Part (b)

Based on the regression table from part (a), is there statistical evidence to suggest that any of the coefficients might be nonzero? Use a 5% level of significance, and justify your answer.

Solution:

Replace this line with your answer.

Answer Check:

There is no autograder for this question; your TA will manually check that your answers are correct.

! Question 4

Use the `model.matrix()` function to extract the model matrix **X** from the `full_mod` relationship. Then, using only basic linear algebra operations in R (e.g. `%*` for matrix multiplication, `t()` for

transpose, and `solve()` for matrix inversion), recompute the OLS estimates for the coefficients of the full model. Check that these agree with the regression table from Question 3. Store your resulting estimates in a vector called `beta_est`.

Solution:

```
## replace this line with your code
```

Answer Check:

```
# DO NOT EDIT THIS LINE
invisible({check("tests/q4.R")})
```

Test q4 failed:

object 'beta_est' not found

 Caution

When specifying the `newdata` argument in the `predict()` function as applied to a model fitted by `lm()`, the new data must:

- 1) be a data frame
- 2) have the same variable names as the original data frame

! Question 5**Part (a)**

A new house has gone on the market. It has 3 bedrooms, lies 1.24 miles from the nearest highway, and lies 0.45 miles from the nearest town center. Use the `full_mod` model to predict the rental price of this apartment, in thousands of dollars per month. Assign this value to a variable called `pred_ppm1`.

Solution:

```
## replace this line with your code
```

Answer Check:

```
# DO NOT EDIT THIS LINE
invisible({check("tests/q5a.R")})
```

Test q5a failed:

object 'pred_ppm1' not found

Part (a)

Another new house has gone on the market. It has 3 bedrooms, lies 10.1 miles from the nearest highway, and lies 0.45 miles from the nearest town center. Explain why it is **not** a good idea to use the `full_mod` model to predict the rental price of this apartment. There is a specific word/term I'm looking for here; also, you should justify your answer using code output. **Hint:** `summary()`.

Solution:

```
## replace this line with your code
```

Answer Check:

There is no autograder for this question; your TA will manually check that your answers are correct.

Part II: Multicollinearity

In this part of the lab, we explore the effects of multicollinearity through simulation.

First, we generate two independent draws of size 1000 from a standard normal distribution:

```
n <- 1000
set.seed(100)
x1 <- rnorm(n); x2 <- rnorm(n)
```

Next, we construct a new variable that is simply the sum of `x1` and `x2`:

```
x3 <- x1 + x2
```

What this means is that `x1`, `x2`, and `x3` are highly collinear! Nevertheless, let's construct a response value `y` as a linear combination of `x1`, `x2`, and `x3`, perturbed by zero-mean and homoskedastic normal noise:

```
y <- x1 + x2 + x3 + rnorm(n)
```

Now, we construct a data frame using `y`, `x1`, `x2`, and `x3`:

```
df1 <- data.frame(y, x1, x2, x3)
```

Here is the correlation matrix of `df1`:

```
cor(df1)
```

	y	x1	x2	x3
y	1.0000000	0.69027267	0.64147554	0.9356903
x1	0.6902727	1.00000000	0.01409256	0.7289175
x2	0.6414755	0.01409256	1.00000000	0.6948059
x3	0.9356903	0.72891747	0.69480590	1.0000000

As expected, it reveals that **x1** and **x2** are not highly correlated between themselves, but are both highly correlated with **x3**.

`lm()` still produces an optimal linear fit:

```
(lm1 <- lm(y ~ . , data = df1))
```

Call:

```
lm(formula = y ~ ., data = df1)
```

Coefficients:

(Intercept)	x1	x2	x3
-0.0139	1.9618	1.9112	NA

but now R is unable to produce an estimate for the parameter associated with **x3**.

! Question 6

Recall that the variance-covariance matrix of the OLS estimators is given by $\sigma^2(\mathbf{X}^T\mathbf{X})^{-1}$. Because our data was simulated, we know to take $\sigma^2 = 1$. Compute the variance-covariance matrix of the OLS estimators in this simulation, and assign this to a matrix called **Sigma1**.

Solution:

```
## replace this line with your code
```

Answer Check:

```
# DO NOT EDIT THIS LINE
invisible({check("tests/q6.R")})
```

Test q6 failed:

```
object 'Sigma1' not found
```

What you should see is that the variances of the OLS estimators is *huge*! This is one of the consequences of multicollinearity - the OLS estimators become highly *unstable*.

Also, returning to the output of `lm()` above: note that the estimated coefficients for **x1** and **x2** are nearly twice what they should be (recall that, when we simulated the data, we set the parameters associated with these two variables to be 1).

This also makes sense, if we think about the model `lm()` is trying to fit.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \text{noise}$$

Since we know $x_3 = x_1 + x_2$ in this case, our model is actually:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 (x_1 + x_2) + \text{noise}$$

which can be rearranged as

$$y = \beta_0 + (\beta_1 + \beta_3) x_1 + (\beta_2 + \beta_3) x_2 + \text{noise}$$

Indeed, `lm()` has estimated $(\beta_1 + \beta_3)$ and $(\beta_2 + \beta_3)$ quite well! The problem is, it is now near impossible to estimate β_1 and β_2 *separately*, as we can only obtain estimates for $(\beta_1 + \beta_3)$ and $(\beta_2 + \beta_3)$. So this is another consequence of multicollinearity - even the parameters we *are* able to estimate have their effects intertwined with the variable(s) that are multicollinear.

In practice, one of the easiest things to do when we detect multicollinearity is to remove the offending variable.

! Question 7

Generate a new fit, called `lm2`, that regresses `y` onto only `x1` and `x2`. Then, display the regression table and compare this to the regression table for `lm1`. Are they at all similar?

Solution:

```
## replace this line with your code
```

Answer Check:

There is no autograder for this question; your TA will manually check that your answers are correct.

Submission Details

Congrats on finishing this PSTAT 100 lab! Please carry out the following steps:

i Submission Details

- 1) Check that all of your tables, plots, and code outputs are rendering correctly in your final `.pdf`.
- 2) Check that you passed all of the test cases (on questions that have autograders). You'll know that you passed all tests for a particular problem when you get the message "All tests passed!".
- 3) Submit **ONLY** your `.pdf` to Gradescope. Make sure to **match ALL pages to the ONE question on Gradescope**; failure to do so will incur a penalty of 0.1 points.